

A Mostly Harmless Introduction to Claude Code, Codex, and Friends

Study guide for empirical researchers

Daniel Sánchez Pazmiño

Overview

This workshop introduces agentic AI for empirical research. The central distinction is between a static chat model, which answers questions inside a chat window, and an agent that can inspect a local project, write files, run commands, and examine the results. The examples focus on data cleaning, statistical code, legacy replication repositories, literature review, reusable instructions, connectors, and local models.

The recording is a practical orientation rather than a complete course in any one software package. The most useful habit it recommends is to keep the researcher in charge: give the agent a bounded task, inspect what it changed, run the analysis yourself, and verify the result against the research question.

Note

The source transcript was generated automatically and lightly cleaned. Product names and code terms have been normalized from context where speech recognition was uncertain. Examples below are adapted from the demonstration and are not presented as verbatim screen captures.

Learning objectives

After studying this guide, you should be able to:

- distinguish a chat model from an agent with access to a project and terminal;
- choose a sensible first workflow for an R, Python, Stata, or mixed research repository;
- write bounded prompts that reduce unnecessary decisions and token use;
- protect confidential data and recognize prompt injection as a security risk;
- use Git, plans, permissions, and project instructions to make agent work recoverable;
- apply an agent to data cleaning, code review, legacy code, replication, and literature search;
- understand when an MCP connector, a reusable skill, or a local model is appropriate.

Prerequisites and scope

The talk assumes basic familiarity with files, a terminal, and at least one research tool such as R, RStudio, Python, Stata, or MATLAB. A local agent is most useful when the corresponding software is installed,

because the agent uses the computer’s own programs to execute code. Some hosted agent features also require a paid subscription.

The examples use a local folder containing compressed Ecuadorian employment-survey files, R scripts, a legacy replication project, a paper in PDF form, and supporting research material. The exact project is less important than the pattern: an agent works best when the source files, instructions, scripts, and outputs are visible in one bounded project.

1. From static chat to an agent

A static model can explain a function, summarize text, or suggest code. It is constrained by what is pasted or uploaded to the chat and usually cannot inspect the actual state of a local research project. An agent adds a loop around the model:

1. inspect files and available tools;
2. propose or write a change;
3. run a command or statistical program;
4. inspect the output;
5. revise the work when the result does not match the request.

That loop is the important change. An agent can read raw data, create a cleaning script, call R from a terminal, and inspect the resulting CSV. It can also organize folders or explain an old repository. A chat model may describe those steps, but it does not automatically perform them on the researcher’s computer.

The talk uses a cake analogy for token limits. A subscription provides a limited allocation of tokens, and more complex prompts and longer files consume more of that allocation. Stronger models can be useful for planning or difficult review, while a faster model is often enough for routine implementation. The practical lesson is to spend context carefully: give the agent the relevant files and a precise task instead of asking it to make every decision from scratch.

An adapted version of the workshop’s first project prompt might look like this:

```
Inspect the files in case_one. They are monthly compressed employment-survey files.  
Create a clean dataset with one row per Ecuadorian province and one column per month.  
Calculate median labour income using the survey weights. Preserve the raw files,  
explain every transformation, and tell me which commands you ran.
```

The prompt defines the input, the unit of observation, the outcome, and a safety constraint. It still leaves room for the agent to inspect the data dictionary and ask about an ambiguity, but it does not ask for an unbounded research project.

2. Security, confidentiality, and recoverability

An online agent may work with local files, but information it reads can still be sent to the provider. The talk therefore advises against using online agents with social-security numbers, health identifiers, personally identifiable information, or data protected by a data-sharing agreement. A local open-source model can keep data on the researcher’s computer, but it requires suitable hardware and a more involved setup.

Prompt injection is a second risk. Untrusted web pages, documents, or copied text can contain instructions aimed at the agent rather than information for the researcher. A request to summarize a page can therefore carry hidden instructions to open files, disclose information, or run a command. Treat external content as data, not as an instruction source, and use only sources you trust.

Agents can also make ordinary mistakes with serious consequences. A mistaken command can overwrite files, delete a folder, or apply a transformation to the wrong data. The talk recommends several layers of protection:

- use Git or another recoverable backup for code and important project files;
- work in a bounded project folder and use a sandbox when available;
- select a plan or manual permission mode for unfamiliar tasks;
- ask the agent to explain what it did and preserve raw inputs;
- run tests and inspect the code before trusting a result.

Warning

An agent's confidence is not evidence that an analysis is correct. The researcher remains responsible for the specification, identification strategy, variable definitions, statistical assumptions, and final interpretation.

3. A safe first workflow in a project folder

The demonstration opens a local folder in Claude Code, points the agent to the workshop repository, and asks it to inspect the first case. The same pattern applies to Codex, Cursor, and GitHub Copilot: open the project, state the task, let the agent inspect before editing, and keep the raw data separate from derived outputs.

The agent can use a graphical application or a terminal. Working from a terminal inside RStudio is useful when the researcher wants to keep the familiar IDE in view. The command shown in the talk is conceptually equivalent to starting the agent from the current project directory:

```
cd path\to\research-project
claude
```

The exact command depends on the installed CLI. The important property is the working directory: the agent should start inside the repository it is meant to inspect.

Three operating modes are especially useful:

- **Automatic mode:** convenient for a routine, well-bounded task when the project is backed up.
- **Manual mode:** the agent asks before running commands or making changes.
- **Plan mode:** the agent writes a proposed sequence without implementing it, which is useful for a large or uncertain change.

The recording also discusses memory. A provider may retain preferences from previous conversations if memory is enabled. This can make later work more consistent, but it can also preserve a mistaken assumption. Inspect, correct, or disable memory when the project requires a clean context.

Project instructions are a more explicit and portable control. Claude Code reads `CLAUDE.md`; other agents commonly read `AGENTS.md`. A small file at the repository root can say, for example:

```
# Project instructions

- Write readable, script-first statistical code.
- Preserve raw data and explain derived files.
- Run the relevant checks before reporting completion.
- Ask before changing the research specification.
- Use the project's preferred language and naming conventions.
```

The talk recommends keeping these instructions close to the project and updating them as new model behaviour creates new problems. They are not a substitute for review, but they reduce repeated corrections.

For code, a minimal recoverability cycle is:

```
git status
git add path/to/intended/files
git commit -m "Save validated analysis change"
```

These commands are a compact illustration of the principle discussed in the talk: create a checkpoint before allowing an agent to make a substantial change, and keep the remote repository available as a recovery path.

4. Data cleaning and statistical code

The first live example begins with monthly compressed survey files from Ecuador. The agent unzips and organizes them, derives a province variable, identifies the income field, notices special or top-coded values, and writes R code to produce a province-by-month dataset. It also computes a survey-weighted median of labour income.

This is valuable because the mechanical work is time-consuming, but the example also shows the limits of automation. The agent made decisions the speaker had not explicitly requested, including details of the weighted calculation. That can be helpful when the choice is correct and dangerous when it is not. The researcher should inspect the data dictionary, the handling of missing and special values, the survey design, and the final sample before using the result.

The recommended division of labour is:

1. ask the agent to inspect and propose a transformation;
2. allow it to write a readable script and a short explanation;
3. open the script in RStudio or the relevant IDE;
4. run it in the actual research environment;
5. compare diagnostics and outputs with the intended design.

The agent can run code through the terminal when R or another program is installed. It is not executing the analysis inside a magical cloud version of R. It is calling the researcher's local installation and then reading the result. This is why a missing R, Stata, or Python installation prevents the workflow from completing.

The talk notes that language models are trained heavily on software-engineering code. They may therefore

produce abstractions, loops, or modules that are technically valid but unnecessarily hard for a researcher to read. Readability is not cosmetic in empirical work: it supports review, replication, and later correction. State the preferred style in the prompt or in project instructions.

5. Legacy repositories, replication, and translation

An agent is often most useful when the code already exists. The speaker points it to an old replication folder and asks for an explanation without running or modifying anything. This is a good first step for a repository that has not been opened for several years, or for a replication package downloaded from a journal.

The agent can summarize the scripts, identify the data flow, locate graphs and side scripts, and explain how the final results are produced. It can also propose a translation from R or Python to Stata. Running Stata requires a valid local installation and, in the talk’s workflow, an additional Stata MCP connector. The connection is described as less seamless than the one for open-source languages, partly because there is less public training material for some closed-source software.

An adapted review prompt is:

```
Review this legacy replication repository. Do not modify files and do not run code.  
Explain the data flow, the main scripts, the outputs, and any assumptions that  
would need to be checked before reproducing the reported results.
```

For a real replication audit, the agent can go further: run the code, compare the reported tables with the generated output, locate inconsistencies, and propose a correction. The talk presents this as a way to make code review and replication checks less expensive. It still requires an expert to verify whether the code implements the paper’s intended specification and whether the data and environment are comparable.

6. Literature review and document formats

The workshop compares three common inputs for a literature task. A Markdown file is the easiest for an agent to read and costs less context than a PDF. A Word document is usually easier than a PDF. PDFs can be read, but layout, columns, tables, and scanned text make extraction more expensive and less reliable.

For a paper already stored in a project, a sensible sequence is:

1. convert the document to a reviewable text format when confidentiality permits;
2. ask the agent to summarize the argument and identify the literature gap;
3. ask it to search for additional papers or use a paper-search connector;
4. inspect the papers and the proposed citations yourself;
5. write only the claims that the sources actually support.

The talk mentions connectors for PubMed, Google Scholar, and other paper-search services. Some require an account or an institutional subscription. Install a connector only from a trusted source and limit it to the current project when the agent offers that choice.

An adapted prompt for this workflow is:

```
Read the paper text in case_three. Summarize the literature review, identify  
unsupported or missing claims, and propose three papers to investigate. Do not
```

```
rewrite the paper yet. Give the reason for each recommendation and distinguish
what you verified from what needs my review.
```

The goal is not to delegate the scholarly judgment. The agent can widen the search and reduce mechanical effort, but it cannot decide whether a contribution is genuinely new or whether a research design is credible.

7. Reusable instructions, skills, and MCPs

The talk separates three kinds of customization:

- **Project instructions:** files such as `CLAUDE.md` or `AGENTS.md` that apply within a repository.
- **Skills:** reusable Markdown instructions for a task such as proofreading, reviewing a paper, humanizing text, creating a lecture, checking a package, or reproducing an author’s voice.
- **MCPs:** connectors that give an agent an additional capability or a structured way to access a source, such as a paper database, a government data service, or Stata.

A skill is best understood as a reusable process description rather than a model or a program. It can specify the inputs, the sequence of checks, the output format, and the quality standard. The speaker describes building a writing-voice skill from several papers and then applying it to later drafts. The same pattern can be used for code conventions or a replication workflow.

The maintenance cost matters. New model releases can introduce new habits, such as overly abstract code or an unnatural writing style. Treat instructions as living project assets: test them on a small task, review the output, and update them when a recurring failure appears. Keep project-specific rules in the project and general rules in a reusable skill.

MCPs can extend an agent beyond its default tools, but they also expand the trust boundary. A connector that reads email, queries a data service, or runs Stata should be enabled only when needed, scoped to the smallest useful project, and reviewed like any other software dependency.

8. Local models and sensitive research

For restricted data, the talk recommends considering a local open-source model rather than sending information to an online provider. A university or employer may host a model for researchers so that data remains within its infrastructure. Individuals can also run smaller models, but the tradeoff is hardware: ordinary computers cannot run the most capable models at comfortable speed.

The key distinction is not that a local model is automatically correct or safe. It changes where the data is processed. You still need access controls, backups, review, and a clear understanding of what software is running. A local model may also be weaker at niche statistical code, so the researcher should test it on representative tasks before relying on it.

Cheat sheet

Need	Agent pattern	Minimal example
Inspect a project	Start in the project root and ask for an inventory	Inspect the repository. Do not modify files.

Need	Agent pattern	Minimal example
Bound a task	State inputs, outputs, constraints, and checks	<code>Preserve raw files and report every transformation.</code>
Protect changes	Use a checkpoint before a substantial edit	<code>git status</code> then a focused commit
Control execution	Choose automatic, manual, or plan mode	<code>Plan the changes; do not implement yet.</code>
Set preferences	Add repository instructions	<code>CLAUDE.md</code> or <code>AGENTS.md</code>
Clean data	Ask for a script and diagnostics, then run it yourself	<code>Write the cleaning script and explain special values.</code>
Understand legacy code	Request an explanation without edits first	<code>Explain the data flow; do not run or modify code.</code>
Review replication	Compare paper claims, code, data, and outputs	<code>Identify mismatches and show how to verify them.</code>
Search literature	Use text input and a scoped paper connector	<code>Propose papers and separate verified claims from leads.</code>
Access structured data	Use a trusted data MCP	<code>Query the official source and save the raw response.</code>
Protect restricted data	Use an approved local or institutional model	<code>Do not send this dataset to an external service.</code>

Final takeaways

Agentic AI is most useful in research when it removes mechanical friction without removing human control. Give it a bounded project, explicit instructions, and a recoverable working state. Let it inspect files, write scripts, run routine commands, explain legacy code, search sources, and produce drafts. Then verify the data, code, specification, citations, and interpretation yourself.

The workshop’s practical sequence is therefore: protect the project, state the task precisely, inspect the agent’s plan, let it work within a defined boundary, run and review the result, and turn repeated preferences into project instructions or reusable skills. For confidential data, move the workflow to an approved local or institutional model rather than assuming that a local folder alone prevents external processing.

Resources named in the recording

The recording discusses Claude Code, Codex, Cursor, GitHub Copilot, RStudio, Git, Stata MCP, paper-search MCPs, PubMed, Google Scholar, government-data MCPs, local open-source models, `CLAUDE.md`, and `AGENTS.md`. These names are retained as the workshop’s resource map; no additional links have been added beyond what appears in the recording.